

ELCT918 – Lab 1 Report

Multi-Objective Design Space Exploration of Neural Network Hyper-Parameters

Team members

Name	ID
Nada Yehia Elsayed	S2701048
Sama Samy Mohamed	S2701051

1. Problem Summary

Neural network architecture has a direct effect on both the implementation cost and the classification accuracy of the network. Increasing the number of hidden layers or the number of nodes per hidden layer generally increases the number of trainable parameters and therefore increases the implementation cost. At the same time, a larger network may achieve better classification accuracy.

Therefore, selecting a neural network architecture can be treated as a multi-objective Design Space Exploration (DSE) problem, where we investigate the trade-off between:

- Implementation cost
- Accuracy drop

The objective of this lab is to generate different fully connected neural networks with different numbers of hidden layers and nodes per hidden layer, train them on the MNIST handwritten-digit dataset, evaluate their performance, and identify the Pareto-optimal configurations.

The assignment describes this problem as a DSE problem because different layer/node combinations can provide different trade-offs between implementation cost and accuracy.

2. Framework and Network Generator

2.1 Framework

The implementation was developed using **TensorFlow/Keras**.

MNIST is loaded directly using:

```
tf.keras.datasets.mnist.load_data()
```

The dataset contains:

- 60,000 training images
- 10,000 testing images
- Each image has a size of 28×28 pixels
- Each image represents one of the digits from 0 to 9

Before training, the pixel values are converted to floating-point values and normalized from the range [0,255] to [0,1].

```
x_train = x_train.astype(np.float32) / 255.0
```

```
x_test = x_test.astype(np.float32) / 255.0
```

2.2 Network Generator

A parameterized function called `build_model()` was implemented:

```
def build_model(num_layers, nodes_per_layer):
```

The two parameters are:

- `num_layers`: number of hidden layers
- `nodes_per_layer`: number of neurons in every hidden layer

This satisfies the assignment requirement that the number of hidden layers and nodes per layer should be genuine parameters rather than manually written for each configuration.

The generated network has the following structure:

MNIST Image

28×28



Flatten



784 inputs



Hidden Layer 1



Hidden Layer 2



...



Hidden Layer n



Output Layer

10 neurons

The Flatten layer converts the 28×28 image into a vector containing 784 pixel values.

The hidden layers use:

activation="relu"

The final output layer contains 10 neurons because MNIST contains 10 possible classes and uses:

activation="softmax"

3. Design Space

The design space was selected according to the range suggested in the assignment.

Number of hidden layers

layer_options = [1, 2, 3]

Therefore:

n = 1, 2, or 3 hidden layers

Nodes per hidden layer

node_options = [10, 20, 40, 80, 160, 200]

This explores nodes from 10 to 200 using approximately logarithmic spacing, as recommended.

Therefore, the complete design space contains:

3 layer configurations × 6 node configurations = 18 configurations

The configurations are:

Hidden Layers	Nodes per Layer
1	10
1	20
1	40
1	80
1	160
1	200
2	10
2	20
2	40
2	80
2	160
2	200
3	10
3	20
3	40
3	80
3	160
3	200

This design space was chosen because it is large enough to observe the effect of both network depth and width while remaining small enough to train all configurations within the available time.

4. Training Configuration

To make the comparison between architectures fair, the same training settings were used for all 18 configurations.

Parameter	Value
Framework	TensorFlow/Keras
Optimizer	Adam
Learning rate	0.001
Epochs	5
Batch size	32
Activation – hidden layers	ReLU
Activation – output layer	Softmax
Dataset	MNIST

Training budget justification

A relatively short training budget of 5 epochs was selected.

The purpose of this experiment is not to train each network until complete convergence. Instead, the objective is to compare the relative behavior of different architectures under the same training conditions.

Using the same number of epochs, batch size, optimizer, and learning rate ensures that every configuration is evaluated under the same training budget.

5. Cost Model

For this experiment, **total number of trainable parameters** was used as the implementation cost.

The parameter count is obtained using:

```
num_parameters = model.count_params()
```

Therefore:

Cost = total number of trainable parameters

For example, a Dense layer connecting N inputs to M neurons contains:

$N \times M$ weights

And M biases

Therefore, its total number of parameters is:

$N \times M + M$

Using parameter count provides a simple and consistent estimate of the network's implementation size.

6. Accuracy Metric

After training each model, its performance is measured using the MNIST test dataset:

```
test_loss, test_accuracy = model.evaluate(  
    x_test,  
    y_test,  
    verbose=0  
)
```

The accuracy drop is then calculated as:

Accuracy Drop (%) = $100 - \text{Test Accuracy \%}$

A lower accuracy drop represents better classification performance.

7. Design Space Exploration Method

An **exhaustive search** was used.

The program loops through every combination of:

layer_options = [1, 2, 3]

node_options = [10, 20, 40, 80, 160, 200]

using nested loops:

for num_layers in layer_options:

 for nodes_per_layer in node_options:

This results in:

$3 \times 6 = 18$ configurations

For every configuration, the program:

1. Generates the network.
2. Compiles the model.
3. Trains the model for 5 epochs.
4. Evaluates the model on the test dataset.
5. Calculates the parameter count.
6. Calculates the accuracy drop.
7. Stores the results.

Since the selected design space contains only 18 configurations, exhaustive exploration was practical and ensures that every configuration in the defined design space is evaluated.

8. Pareto-Optimal Front

After evaluating all configurations, the program determines the Pareto-optimal configurations.

A configuration **A dominates configuration B** if:

- A has a cost that is no greater than B's cost.
- A has an accuracy drop that is no greater than B's accuracy drop.
- A is strictly better in at least one of these objectives.

Therefore, a configuration is Pareto-optimal if no other explored configuration dominates it. The implemented function is:

```
def find_pareto_front(results):
```

It compares every configuration against the other configurations and removes configurations that are dominated.

9. Results

FINAL DSE RESULTS

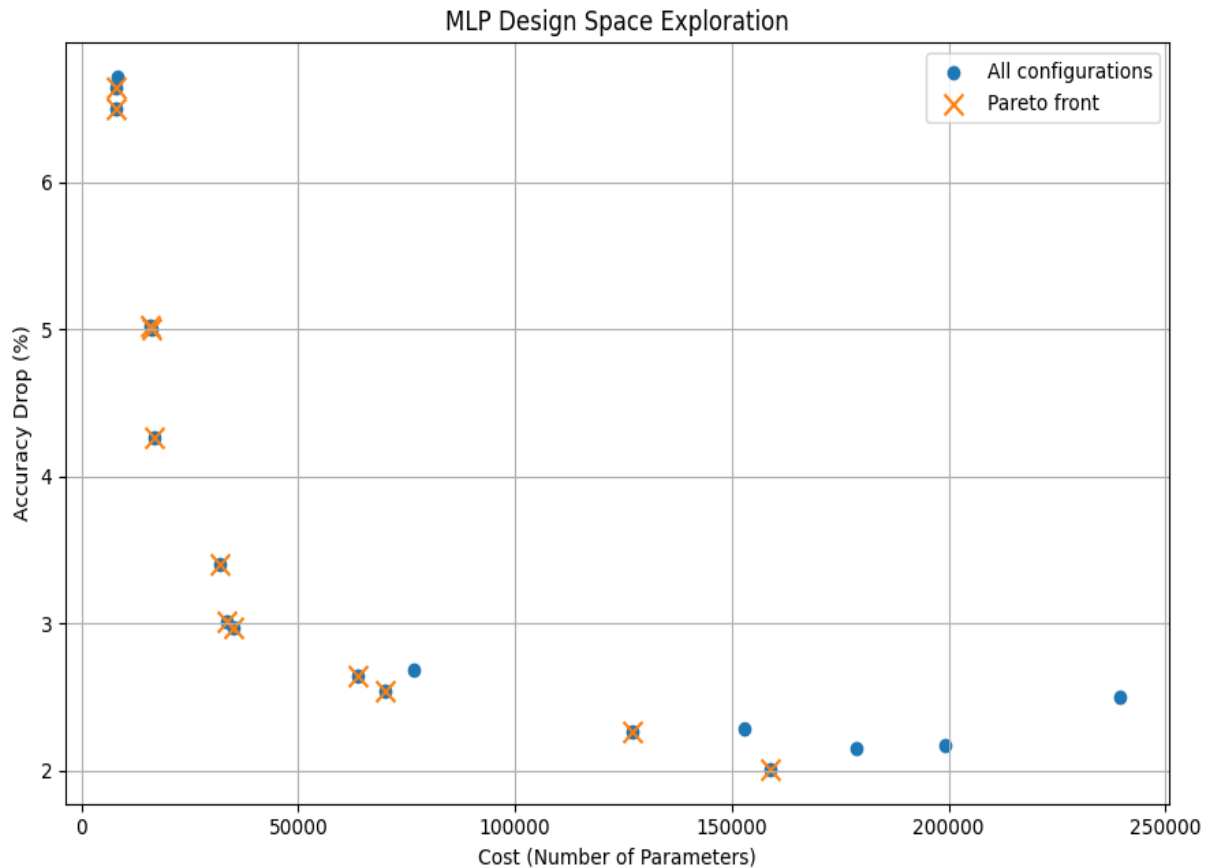
Layers	Nodes	Parameters	Accuracy	Accuracy Drop
1	10	7,960	93.36%	6.64%
1	20	15,910	94.98%	5.02%
1	40	31,810	96.60%	3.40%
1	80	63,610	97.36%	2.64%
1	160	127,210	97.74%	2.26%
1	200	159,010	97.99%	2.01%
2	10	8,070	93.50%	6.50%
2	20	16,330	95.00%	5.00%

2	40	33,450	96.99%	3.01%
2	80	70,090	97.46%	2.54%
2	160	152,970	97.72%	2.28%
2	200	199,210	97.83%	2.17%
3	10	8,180	93.29%	6.71%
3	20	16,750	95.74%	4.26%
3	40	35,090	97.03%	2.97%
3	80	76,570	97.32%	2.68%
3	160	178,730	97.85%	2.15%
3	200	239,410	97.50%	2.50%

PARETO-OPTIMAL CONFIGURATIONS

Layers	Nodes	Parameters	Accuracy	Accuracy Drop
1	10	7,960	93.36%	6.64%
2	10	8,070	93.50%	6.50%
1	20	15,910	94.98%	5.02%
2	20	16,330	95.00%	5.00%
3	20	16,750	95.74%	4.26%
1	40	31,810	96.60%	3.40%
2	40	33,450	96.99%	3.01%
3	40	35,090	97.03%	2.97%
1	80	63,610	97.36%	2.64%
2	80	70,090	97.46%	2.54%
1	160	127,210	97.74%	2.26%
1	200	159,010	97.99%	2.01%

10. Pareto Front Visualization



The expected interpretation is that moving toward the right generally represents a higher parameter cost, while moving downward represents a lower accuracy drop.

The Pareto front represents the configurations for which improving one objective would require accepting a degradation in the other objective, within the explored design space.

11. Discussion of Trade-offs

The results show a clear trade-off between model cost and accuracy. Increasing the number of nodes generally increases the parameter count while reducing the accuracy drop.

For example, for the 1-layer networks, increasing nodes from 10 to 200 increased parameters from 7,960 to 159,010, while reducing the accuracy drop from 6.64% to 2.01%.

Adding hidden layers can further reduce the accuracy drop, but also increases the parameter count. For example, with 40 nodes, the accuracy drop decreases from 3.40% (1 layer) to 2.97% (3 layers), while parameters increase from 31,810 to 35,090.

However, increasing the network size does not necessarily result in a proportional improvement in test accuracy. Some larger configurations achieve only small improvements or even lower accuracy .

For example:

- 2 layers, 200 nodes → **97.83%**
- 3 layers, 200 nodes → **97.50%**

This is why a simple selection based only on minimum cost or maximum accuracy does not fully describe the DSE problem. Instead, the Pareto front identifies the configurations that represent the non-dominated trade-offs between the two objectives.

Repository link:

<https://github.com/Nada44yehia/ELCT918.git>